

Jour 2 — Étage hors-contexte

Grammaires & automate à pile

Modules : `pda.py`, `cfg.py` • App : `shield/delimiters` • Barème : 4 pts (+1)

Cadre éthique

$\Sigma_3 = \{a, o, r, [,], (,)\}$. Aucun contenu réel.

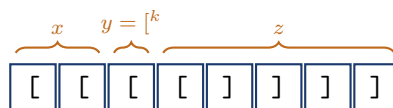
Contexte

Les **faux délimiteurs** imitent un bloc `[ ... ]` ou un cadre `( ... )`, éventuellement imbriqués. Vérifier l'imbrication bien formée = reconnaître un langage de type **Dyck** : c'est **hors-contexte**, hors de portée d'un automate fini. Le nouvel ingrédient : la **pile**.

Notions nouvelles du jour

1. Le lemme de l'étoile (pompage)

Outil pour **prouver qu'un langage n'est pas régulier**. Si L est régulier, il existe p tel que tout mot long se coupe $w = xyz$ ($|xy| \leq p$, $y \neq \varepsilon$) avec $xy^iz \in L$ pour tout i . Pour $D = \{[n]^n\}$, prendre $w = [p]^p$: comme $|xy| \leq p$, le bloc y n'est fait que de `[` ; alors $xy^2z = [p+k]^p \notin D$.



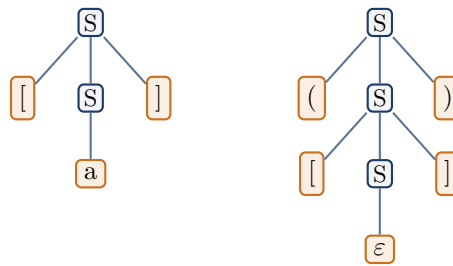
Le déséquilibre casse l'appartenance : contradiction. Un automate fini a une mémoire **bornée**, il ne peut pas « compter » les ouvrantes.

2. Grammaire hors-contexte et arbre de dérivation

Une CFG engendre par **réécriture**. Ici :

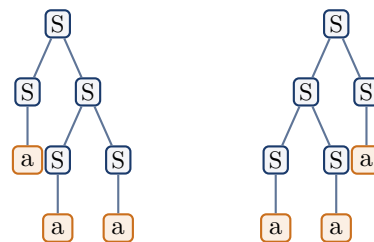
$$S \rightarrow SS \mid [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon.$$

L'**arbre de dérivation** montre *comment* un mot est produit. Exemple pour `[a]` (à gauche) et `([])` (à droite) :



3. Ambiguïté

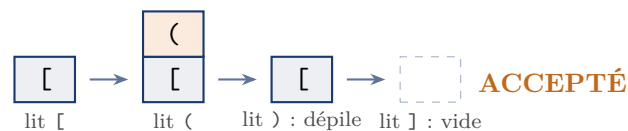
Une grammaire est **ambiguë** si un mot admet *plusieurs* arbres de dérivation. La règle $S \rightarrow S S$ n'est pas associée : `aaa` a (au moins) **deux** arbres :



On désambiguïse en imposant l'associativité : $S \rightarrow T S \mid T ; T \rightarrow [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$.

4. La pile : mémoire non bornée mais disciplinée

Contrairement à l'état fini d'un AFD, la **pile** (LIFO) est **non bornée**, mais on n'y accède qu'au **sommet**. Elle suit la **profondeur d'imbrication**. Ci-dessous, l'évolution sur `[a(r)]` (on ignore `a,r`) : accepté car pile **vide** à la fin.



Un rejet : `([)]`

On empile `(` puis `[`. Sur `)`, le sommet est `[` $\neq$ ouvrante attendue $(\Rightarrow$ **rejet immédiat**. L'imbrication « croisée » est mal formée : c'est bien la *pile* (LIFO) qui l'attrape, pas un simple compteur. • `DelimiterPDA.accepts` (test `test_delimiters_pda`).

CFG $\leftrightarrow$ PDA, langage de Dyck

Grammaire (génère) et automate à pile (reconnait) décrivent la **même** classe : les langages **algébriques**. Le langage de **Dyck** (parenthésages bien formés) est

l'exemple canonique d'un langage hors-contexte *non* régulier.

A. Théorie (à rédiger) — 2,5 pts

Q2.1 (1,5)

Montrer par pompage que $D = \{[n]^n\}$ n'est pas régulier ; conclure que l'AFD ne peut vérifier l'imbrication.

Q2.2 (0,5)

Grammaire G des prompts bien parenthésés sur Σ_3 .

Q2.3 (0,5)

G est-elle ambiguë ? Deux arbres pour `aaa` ; version non ambiguë.

B. Pratique — 1,5 pt

E2.1 (1)

`well_parenthesized` avec une pile (`DelimiterPDA`). Test : `test_delimiters_pda`.

E2.2 (0,5)

`CFG.generate(max_len)`. Test : `test_cfg_engendre_des_mots_equilibres`.

Piège : avec $S \rightarrow SS \mid \dots \mid \varepsilon$, les formes purement non-terminales (`S`, `SS`, `SSS`...) ont 0 terminal et ne sont jamais élaguées par la seule borne sur les terminaux $\Rightarrow$ explosion. Borner **aussi** le nombre de non-terminaux (cf. corrigé).

Mini-barème

Item	Détail	Pts
Q2.1	pompage $\{[n]^n\}$	1,5
Q2.2	grammaire bien parenthésée	0,5
Q2.3	ambiguïté + désambiguïsation	0,5
E2.1	PDA <code>well_parenthesized</code>	1
E2.2	génération bornée	0,5
Bonus	reconnaissance par CYK	+1

Gate (règle non négociable)

`delimiters.py` instancie `formlang.pda.DelimiterPDA`.

Références

Hopcroft, Motwani, Ullman — lemme de l'étoile, CFG, PDA, ambiguïté ; langage de Dyck.